

《交通大数据技术》课程总结复习题解答

根据课程总结与第 1-6 章课件整理

1 第一章：交通大数据技术概述

1.1 数据与信息的含义

数据 数据是对客观事物的性质、状态、数量、关系等进行记载的物理符号或符号组合。它可以是数字、文字、图像、音频、视频、地理坐标、传感器读数等。数据本身是较为原始和离散的记录，只有经过组织、加工和解释后才更容易产生使用价值。

信息 信息是经过加工、整理、组织并能表达特定含义的数据。数据是信息的基本单位，信息则强调数据所传达的意义。例如，道路检测器采集到的车速数值是数据，将多个车速数据按路段和时间汇总后得到“某路段早高峰拥堵”的判断，就是信息。

1.2 常见数据类型

从计算机处理和数据组织角度看，常见数据类型可分为以下几类：

- **结构化数据**：具有固定字段、固定模式，通常存放在关系型数据库表中，如车辆编号、路段编号、时间、速度、流量等。
- **半结构化数据**：具有一定结构但不完全符合关系表模式，如 XML、HTML、JSON、日志文件等。
- **非结构化数据**：没有固定结构，需借助文本挖掘、图像处理、语音识别等技术解析，如图片、视频、文本、音频等。
- **按数值属性划分**：还可分为数值型、字符型、日期时间型、布尔型、地理空间型等。

1.3 数据使用流程

数据使用通常不是直接从原始数据到结论，而是经历一个完整的数据链条：

1. **数据采集**：从传感器、互联网、日志、业务系统等来源获取数据。
2. **数据存储**：将数据保存到数据库、数据仓库、分布式文件系统或数据湖中。

3. **数据清洗与转换：**处理缺失值、异常值、重复值，统一格式和类型，进行规范化、聚合、脱敏等。
4. **数据分析与挖掘：**利用统计分析、机器学习、深度学习或规则模型发现规律。
5. **数据可视化与应用：**把分析结果转化为图表、报告、预警、决策支持或交通管理服务。

1.4 数据仓库与 ETL

数据仓库 数据仓库是面向主题、集成、相对稳定、反映历史变化的数据集合，主要用于数据分析和决策支持。它通常将来自多个业务系统的数据统一整合，服务于商业智能、统计分析和长期趋势研判。

ETL ETL 是数据仓库建设中的关键流程，包含抽取、转换、加载三个环节：

- **Extract，抽取：**从业务数据库、日志、接口、文件等数据源获取数据。
- **Transform，转换：**对数据进行清洗、格式统一、字段映射、单位换算、缺失值处理、异常值处理等。
- **Load，加载：**将转换后的数据写入数据仓库、数据库或分析平台。

在交通场景中，ETL 可用于把卡口、公交刷卡、浮动车、道路传感器、天气和事件数据统一到交通运行分析平台中。

1.5 大数据的 4V 特征

- **Volume，量大：**数据规模巨大，传统单机存储和处理方式难以承载。
- **Variety，型多：**数据类型多样，既包括结构化数据，也包括图像、视频、文本、轨迹等非结构化或半结构化数据。
- **Velocity，速快：**数据产生、传输和处理速度快，很多交通数据具有实时或准实时特征。
- **Value，值高或价值密度低：**大数据蕴含较高应用价值，但单条数据价值有限，需要通过大规模汇聚和分析发现规律。

1.6 大数据发展的基础

大数据的发展依赖多方面技术条件：

- **存储能力提升：**存储设备容量增加、单位存储成本下降，使海量数据长期保存成为可能。
- **计算能力提升：**CPU、GPU、并行计算和分布式计算提高了大规模数据处理能力。

- **网络带宽增加：**高速网络支撑数据在设备、平台和云端之间传输。
- **相关技术生态成熟：**数据库、数据仓库、Hadoop、Spark、云计算、物联网、人工智能等技术共同构成大数据生态。

1.7 交通大数据的分类与典型数据

交通大数据可以从多个维度进行分类：

- **按交通要素：**人、车、路、环境、管理与服务等。例如车辆轨迹、乘客出行记录、道路状态、天气数据、交通事件数据。
- **按与交通管理和服务的关联度：**基础交通数据、运行状态数据、交通事件数据、出行服务数据、外部关联数据等。
- **按数据类型：**结构化数据、半结构化数据和非结构化数据。
- **按产生与变化频率：**静态数据、准静态数据、动态数据和实时数据。

典型交通大数据包括卡口过车数据、车牌识别数据、浮动车 GPS 轨迹、公交 IC 卡或移动支付数据、地铁刷卡数据、道路检测器流量速度数据、视频监控数据、交通事故和拥堵事件数据、地图路网数据、信号控制数据、停车数据、共享单车与网约车订单数据等。

2 第二章：大数据相关支撑技术

2.1 什么是并行计算

并行计算是把一个较大的计算任务拆分为多个可以同时执行的子任务，并利用多个处理器、多个计算核心或多台计算机协同完成计算的方法。其核心思想是“分而治之”，通过任务划分和并行执行缩短处理时间。

在大数据处理中，并行计算常用于海量日志统计、交通流量计算、轨迹匹配、模型训练等任务。MapReduce、Spark 等框架都体现了并行计算思想。

2.2 计算机网络的功能

计算机网络的基本功能包括：

- **数据通信：**实现不同设备之间的数据传输。
- **资源共享：**共享硬件、软件、数据库、存储和计算资源。
- **分布式处理：**让多台机器协同完成复杂任务。
- **提高可靠性：**通过多节点、多路径和冗余机制提高系统可用性。
- **支持远程服务：**支撑云计算、物联网、远程监控和交通平台联网运行。

2.3 云计算、云计算模式与商业模式

云计算 云计算是把计算、存储、网络、平台和软件等 IT 资源集中起来，通过网络按需提供服务的计算模式。用户不必自建完整机房，而可以像使用水、电一样租用云端资源。

服务模式 常见云计算服务模式包括：

- **IaaS，基础设施即服务：**提供虚拟机、存储、网络等基础资源。
- **PaaS，平台即服务：**提供数据库、中间件、运行环境、开发部署平台。
- **SaaS，软件即服务：**直接通过网络使用应用软件，如在线办公、交通管理平台等。

部署类型与商业模式 云计算部署类型包括公有云、私有云、混合云等。商业模式通常体现为按需计费、弹性扩展、租用服务、平台开放和托管运维。对交通大数据而言，云计算提供了弹性存储和计算能力。

2.4 物联网及其典型架构

物联网 物联网是通过传感器、射频识别、定位系统、红外感应、激光扫描等设备，把物理世界中的对象接入网络，实现识别、感知、通信、控制和智能服务的技术体系。

关键技术 物联网关键技术包括识别与感知技术、网络与通信技术、数据挖掘与融合技术、边缘计算与云平台技术等。

典型架构 物联网典型架构可概括为：

1. **感知层：**通过传感器、摄像头、RFID、GPS 等采集数据。
2. **网络层：**通过有线、无线、蜂窝网络、车联网等传输数据。
3. **平台层：**完成数据管理、设备管理、计算分析和 service 支撑。
4. **应用层：**面向具体业务提供应用，如智能交通、智慧停车、公交调度等。

2.5 边缘计算

边缘计算是把计算、存储和智能分析能力从中心云下沉到靠近数据源或用户的一侧，如路侧设备、车载设备、边缘网关、摄像头前端服务器等。它的优势是降低网络传输压力、减少时延、提高实时响应能力，并增强本地自治能力。

在交通系统中，边缘计算可用于路口视频结构化识别、信号控制实时优化、车路协同安全预警、隧道或高速公路边缘监测等。

2.6 人工智能的关键核心

人工智能的关键核心在于让机器具备感知、学习、推理、决策和行动能力。其基础包括数据、算法和算力：

- **数据：**提供训练和推断的样本基础。
- **算法：**包括机器学习、深度学习、搜索、推理、优化等方法。
- **算力：**支撑大规模模型训练和实时推断。

交通领域中的 AI 应用包括交通状态预测、路径规划、自动驾驶感知、交通事件识别、信号控制优化等。

2.7 知识图谱的基本结构

知识图谱是一种用图结构组织知识的方法，基本单位通常是三元组：

(实体, 关系, 实体)

例如：

(道路 A, 连接, 交叉口 B)

知识图谱由实体、关系、属性和语义约束组成，能够描述对象之间的关联。交通知识图谱可用于组织道路、车辆、事件、地点、设施、规则之间的关系，辅助交通问答、事件推理和决策支持。

2.8 人工智能的三种学习方法

- **监督学习：**训练数据含有输入和正确标签，模型学习从输入到标签的映射，如交通流量预测、拥堵状态分类。
- **无监督学习：**训练数据没有标签，模型自动发现结构和规律，如轨迹聚类、异常出行模式发现。
- **强化学习：**智能体通过与环境交互，根据奖励信号学习最优策略，如交通信号控制、路径决策。

2.9 人工智能学习模式

人工智能的一般学习模式可以概括为：从环境或数据中获取样本，经过特征表示和模型训练形成知识或策略，再将模型用于预测、分类、生成或决策，并通过反馈继续优化。其过程通常包括数据准备、特征构造、模型训练、验证评估、部署应用和反馈更新。

2.10 区块链

区块链是一种分布式账本技术。它把交易数据按时间顺序打包成区块，并通过密码学哈希把区块连接成链。每个节点都可以保存账本副本，账本更新依赖共识机制完成，因此具有去中心化、可追溯、防篡改等特点。

区块链的核心包括区块链数据结构、密码学哈希、数字签名、P2P 网络和共识机制。

2.11 公钥与私钥

公钥和私钥是非对称密码体系中的一对密钥：

- **私钥：**由用户自己保存，用于签名或解密，不能泄露。
- **公钥：**可以公开分发，用于验证签名或加密。

在区块链交易中，用户用私钥对交易信息签名，其他节点用该用户公钥验证签名，从而确认交易确实由私钥持有者发出，并保证交易内容未被篡改。

2.12 中心化记账的缺点

中心化记账依赖单一中心机构维护账本，主要缺点包括：

- 中心节点可能成为单点故障，系统可靠性受限。
- 中心机构拥有较大控制权，存在篡改、作恶或滥用风险。
- 跨机构协同时信任成本较高，数据共享和对账成本较大。
- 一旦中心系统被攻击，账本安全和服务连续性都会受到影响。

2.13 去中心化记账流程

去中心化记账的大致流程如下：

1. 用户发起交易，并使用私钥生成数字签名。
2. 交易广播到 P2P 网络中的各个节点。
3. 节点验证交易合法性，如签名是否正确、余额或状态是否满足要求。
4. 记账节点将一批合法交易打包成候选区块。
5. 网络通过共识机制决定哪个区块被接受。
6. 合法区块被加入区块链，所有节点更新账本副本。

2.14 POW 的含义、作用和工作过程

含义 POW 是 Proof of Work，即工作量证明机制。它要求记账节点通过消耗计算资源求解一个满足条件的数学问题，以争夺记账权。

主要作用 POW 的作用是分配记账权、提高作恶成本、帮助去中心化网络达成共识，并降低恶意节点随意篡改账本的可能性。

工作过程 记账节点将交易打包后，不断尝试不同随机数，使区块哈希值满足难度要求。某节点率先找到满足条件的解后向全网广播区块，其他节点验证区块和工作量。如果验证通过，该区块被加入链中，网络进入下一轮记账竞争。

2.15 大数据与云计算、物联网、区块链之间的关系

- **大数据与云计算：**云计算为大数据提供弹性存储和计算资源，大数据应用又为云计算创造使用需求。
- **大数据与物联网：**物联网是大数据的重要来源，传感器和联网设备持续产生海量实时数据；大数据技术则为物联网提供数据管理、分析和智能决策能力。
- **大数据与区块链：**区块链可为数据确权、可信共享、追溯审计提供机制；大数据分析可挖掘链上链下数据价值。
- **整体关系：**物联网负责感知和采集，网络负责传输，云计算负责存储和计算，大数据负责分析和挖掘，区块链增强可信协同，人工智能提升预测与决策能力。

3 第三章：数据采集、清洗、转换与脱敏

3.1 大数据数据采集的三大要点

大数据采集需要重点关注：

- **数据源广泛：**数据可能来自传感器、互联网、日志文件、企业业务系统、开放接口等。
- **数据量巨大且类型多样：**采集对象既包括结构化业务表，也包括轨迹、图片、视频、网页、日志等。
- **实时性与可用性要求高：**部分场景需要实时或准实时采集，同时要保证数据质量、完整性和后续可处理性。

3.2 数据采集的主要来源

课程中提到的主要数据源包括：

- **传感器数据：**检测器、摄像头、GPS、RFID、雷达、地磁、气象传感器等。

- **互联网数据**：网页、公开 API、社交媒体、地图服务、出行服务平台等。
- **日志文件**：服务器日志、系统日志、应用日志、设备运行日志等。
- **企业业务系统数据**：数据库中的订单、交易、车辆、用户、调度、收费等业务记录。

3.3 订阅者模式

订阅者模式也可理解为发布-订阅模式。生产者把消息发布到某个主题，消费者订阅感兴趣的主体并实时消费消息，中间由消息系统负责缓存、分发和解耦。

以 Kafka 为例，其架构通常包括主题、生产者、服务代理和消费者。该模式适用于高吞吐、实时性较强的数据采集，例如车辆位置流、交通事件流、日志流等。

3.4 网络爬虫基本结构

网络爬虫是自动提取网页的程序。通用网络爬虫的基本结构通常包括：

- 初始 URL 集合；
- URL 队列；
- 页面爬取模块；
- 页面分析模块；
- 链接过滤模块；
- 页面数据库或数据存储模块。

基本过程是从入口 URL 开始下载页面，解析页面中的链接，将符合条件的新 URL 加入队列，并不断递归抓取。

3.5 网络爬虫的类型及工作流程

类型 网络爬虫主要包括：

- **通用网络爬虫**：面向整个 Web 抓取，常用于搜索引擎。
- **聚焦网络爬虫**：只抓取与预定义主题相关的页面。
- **增量式网络爬虫**：只抓取新增或发生变化的页面，保持本地页面库更新。
- **深层网络爬虫**：抓取传统静态链接难以直接到达的深层网页。

工作流程 爬虫一般先确定种子 URL，然后下载网页、解析内容和链接、过滤无关链接、更新 URL 队列、保存结构化结果，并根据调度策略继续抓取。常用爬行策略包括深度优先、广度优先、基于内容评价、基于链接结构评价和 PageRank 优先等。

3.6 缺失值处理方法

缺失值处理方法包括：

- **估算填补**：用均值、中位数、众数或模型估计值填补缺失值，简单但可能引入误差。
- **整例删除**：删除含缺失值的样本，适合缺失比例较小或关键变量缺失的情况。
- **变量删除**：当某变量缺失严重且对研究不重要时删除该变量。
- **成对删除或特殊编码**：用特殊码标记缺失，并在具体分析时选择完整数据。

选择方法时要结合业务含义、数据规模、缺失比例和分析目标。

3.7 异常值处理方法

异常值是超出合理范围或不符合业务逻辑的数据。处理方法包括：

- 根据取值范围和变量间逻辑关系识别异常值。
- 对明显错误的数据进行修正、替换或删除。
- 对可能代表真实特殊事件的数据保留并单独标记。
- 使用分箱、回归、聚类等方法进行平滑或异常检测。

在交通数据中，异常速度、异常定位点、重复过车记录、传感器故障值都需要结合业务规则判断，不能机械删除。

3.8 数据转换和平滑处理方法

常见数据转换策略包括平滑处理、聚集处理、数据泛化、规范化处理和属性构造。

平滑处理 平滑处理用于减少数据噪声，常见方法包括：

- **分箱**：将排序后的数据分配到若干箱中，用箱均值、中位数或边界值替换箱内数据。
- **回归**：用拟合函数描述数据趋势，适合时间序列等连续数据。
- **聚类**：通过聚类发现异常点或用簇中心代表局部数据特征。

3.9 规范化方法

规范化是把不同量纲或不同范围的数据缩放到可比较尺度。常见方法包括：

- **Min-Max 规范化**：

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

常把数据缩放到 $[0, 1]$ 区间，但当新数据改变最大值或最小值时需要重新计算。

- **Z-Score 规范化：**

$$z = \frac{x - \mu}{\sigma}$$

其中 μ 为均值， σ 为标准差，可消除量纲差异并反映相对偏离程度。

- **小数定标规范化：**通过移动小数点位置缩放数据，移动位数取决于属性绝对值最大值。

3.10 GAN 网络结构及训练过程

GAN 即生成对抗网络，由生成器和判别器组成：

- **生成器：**从随机噪声或条件输入生成尽可能接近真实数据的样本。
- **判别器：**判断输入样本是真实数据还是生成器伪造的数据。

训练过程是一个对抗优化过程：生成器努力欺骗判别器，判别器努力区分真实样本和生成样本。经过多轮迭代，生成器逐渐学会生成与真实数据分布相近的数据。GAN 可用于数据生成、隐私保护和数据脱敏，但需要注意生成数据的真实性、可控性和隐私泄露风险。

4 第四章：大模型与 AI Agent

4.1 大模型幻觉及常见解决方法

大模型幻觉 大模型幻觉是指模型生成了看似合理但事实上错误、缺少依据或与上下文不一致的内容。其原因可能包括训练数据缺失、模型过度补全、上下文不足、推理链错误和输出置信度与真实性不匹配。

常见解决方法

- **检索增强生成，RAG：**为模型提供与问题相关的外部知识和上下文。
- **上下文工程：**优化提供给模型的背景、约束、示例、工具结果和历史反馈。
- **工具调用：**让模型调用数据库、搜索、计算器、代码执行器等外部工具核验结果。
- **自我反馈与多轮修正：**先生成草稿，再进行批判性检查和针对性修正。
- **人工审核与边界约束：**对高风险任务设置人工确认、权限边界和输出格式约束。

4.2 Prompt Engineering、Context Engineering 与 Harness Engineering

- **Prompt Engineering：**关注“问什么”，通过优化提示词、角色、任务说明、输出格式和示例，提高单次生成质量。
- **Context Engineering：**关注“传什么”，为模型提供合适的上下文，包括 RAG 检索内容、历史经验、任务状态、约束条件和工具结果。

- **Harness Engineering:** 关注“怎么动”，构建模型运行环境、工具链、权限控制、反馈机制和自愈循环，使模型从一次性问答扩展为可协作、可执行、可修正的系统。

三者关系可以理解为：Prompt 优化输入表达，Context 优化知识和状态供给，Harness 优化模型与环境的协作方式。

4.3 RAG

RAG 是 Retrieval-Augmented Generation，即检索增强生成。它的基本思想是在模型回答前，先从知识库、文档库、数据库或互联网中检索与问题相关的资料，再把检索结果作为上下文提供给大模型生成答案。

RAG 的典型流程包括：

1. 将文档切分、向量化并建立索引。
2. 用户提出问题后，将问题向量化。
3. 从知识库中检索最相关的片段。
4. 把检索片段与问题一起输入大模型。
5. 由模型基于外部资料生成答案，并可附带来源。

RAG 的作用是降低幻觉、引入最新或私有知识、增强可追溯性，但检索质量、切片策略和上下文组织会直接影响效果。

4.4 Agent 与 LLM 的区别

LLM 主要是语言模型，本质上根据输入上下文生成文本、代码或结构化输出。Agent 则是 LLM 的进阶形态，具有目标驱动、任务拆解、工具调用、环境反馈和自我修正能力。

区别可概括为：

- LLM 更像一次性生成器，Agent 更像能围绕目标持续行动的智能体。
- LLM 主要依赖内部参数和当前上下文，Agent 可调用外部工具、读取环境、写入结果。
- LLM 通常被动响应输入，Agent 能根据高层目标主动规划步骤并迭代执行。
- Agent 的运行闭环是 Goal-Plan-Action-Observation-Correction。

4.5 Agent 的三个关键能力

课程中强调 Agent 的关键能力可以概括为三大支柱：

- **环境感知与修正：**能够识别错误输出，利用反馈信息进行第二次尝试，并通过上下文工程规避已知风险。

- **外部工具调用**：当内部知识不足时，能够通过 API、代码执行、检索、数据库查询等方式获得外部能力。
- **任务拆解与动态规划**：能把模糊复杂的目标拆成明确子任务，并根据执行结果动态调整后续计划。

4.6 Agent 如何分阶段实现自我修正

Agent 的自我修正通常不是一次完成，而是分阶段迭代：

1. **初始生成, Draft**：根据目标生成初步方案、代码或答案。
2. **自我反馈, Self-Critique**：检查输出是否满足要求，识别逻辑错误、格式错误、事实错误或执行失败。
3. **针对性修正, Refinement**：根据反馈修改方案，并在必要时重新调用工具或补充上下文。
4. **循环迭代**：在 Plan-Act-Observe-Correct 循环中持续改进，直到达到目标或触发人工介入。

自我修正的局限在于：如果模型初始判断高度自信但错误，可能出现“自我辩护”；对于知识库中完全不存在的信息，模型也难以凭空纠正。因此，外部验证和工具反馈很重要。

5 第五章：Hadoop、HDFS、MapReduce 与 Spark

5.1 什么是 HDFS

HDFS 是 Hadoop Distributed File System，即 Hadoop 分布式文件系统。它把大文件切分为多个数据块，分散存储到集群中的多个数据节点上，并通过多副本机制提供容错能力。

HDFS 适合存储海量大文件，面向高吞吐量数据访问，不适合大量小文件和低延迟随机读写场景。

5.2 什么是 MapReduce

MapReduce 是一种分布式并行编程模型及其实现框架，用于处理和生成大规模数据集。它把数据处理抽象为两个核心阶段：

- **Map**：对输入分片并行处理，生成中间键值对。
- **Reduce**：对相同键的中间结果进行汇总、统计或合并，生成最终结果。

MapReduce 的基本思想是分而治之，让多个节点并行处理数据，并尽量让计算靠近数据。

5.3 HDFS 的几种特性

HDFS 的主要特性包括：

- **分布式存储**：文件被划分为多个块，存储在不同 DataNode 上。
- **高容错性**：通过多副本冗余应对硬件故障。
- **高吞吐量**：适合大规模批处理和顺序读写。
- **可扩展性**：可通过增加节点扩展存储和计算能力。
- **透明性**：用户通过客户端访问文件时，通常无需感知底层分布式细节。

5.4 Hadoop 1.0 与 2.0 的区别

Hadoop 1.0 的核心是 HDFS 和 MapReduce，MapReduce 同时承担计算框架和资源调度职责，扩展性和灵活性受限。Hadoop 2.0 引入 YARN，将资源管理与计算模型解耦，使 MapReduce、Spark、Tez 等多种计算框架可以共享集群资源。

可概括为：

- Hadoop 1.0: HDFS + MapReduce，JobTracker 同时负责资源管理和作业调度。
- Hadoop 2.0: HDFS + YARN + 多计算框架，资源管理更通用，生态扩展能力更强。

5.5 Hadoop 集群中的主要节点类型

Hadoop 集群中的主要节点包括：

- **NameNode**：管理 HDFS 命名空间和元数据。
- **DataNode**：实际存储数据块，并定期向 NameNode 汇报状态。
- **SecondaryNameNode**：辅助 NameNode 合并 FsImage 和 EditLog，并保存部分检查点信息。
- **JobTracker**：在 Hadoop 1.0 MapReduce 中负责资源监控和作业调度。
- **TaskTracker**：执行 JobTracker 分配的 Map Task 或 Reduce Task，并通过心跳汇报状态。

5.6 HDFS 的目标

HDFS 的目标是为大数据应用提供可靠、可扩展、高吞吐的分布式文件存储系统。它强调：

- 支持大文件和海量数据存储；
- 在廉价硬件上运行并容忍硬件故障；

- 通过多副本机制保证可靠性;
- 为批处理任务提供高吞吐读写;
- 为上层 MapReduce、Spark 等计算框架提供数据基础。

5.7 NameNode 与 DataNode 的区别

NameNode 是管理节点, 保存文件系统命名空间、目录树、文件到块的映射、块所在 DataNode 等元数据信息。它不直接保存用户文件内容。

DataNode 是工作节点, 负责保存真实数据块, 处理客户端或 NameNode 调度的读写请求, 并定期发送心跳和块报告。

简言之, NameNode 管“文件在哪里”, DataNode 存“文件内容本身”。

5.8 FsImage 与 EditLog 的作用

- **FsImage**: 保存 HDFS 文件系统树以及文件、目录、权限等元数据的持久化快照。
- **EditLog**: 记录文件系统元数据的增量修改操作, 如创建、删除、重命名文件等。

NameNode 启动时会加载 FsImage, 并回放 EditLog 以恢复最新命名空间状态。

5.9 NameNode 与 SecondaryNameNode 的关系

SecondaryNameNode 不是 NameNode 的实时热备节点。它的主要作用是定期从 NameNode 获取 FsImage 和 EditLog, 将二者合并生成新的检查点文件, 再返回给 NameNode, 以减少 EditLog 过大带来的恢复开销。

如果 NameNode 失效, SecondaryNameNode 上的检查点可辅助恢复部分元数据, 但不能完全替代高可用架构中的备用 NameNode。

5.10 HDFS 采用的架构

HDFS 采用主从式 Master/Slave 架构。一个 NameNode 作为中心服务器管理命名空间和元数据, 多个 DataNode 作为工作节点保存数据块。客户端先向 NameNode 查询元数据, 再直接与 DataNode 进行数据读写。

这种架构简化了元数据管理, 但 NameNode 的可靠性和性能对整个 HDFS 至关重要。

5.11 HDFS 如何实现冗余数据存储、数据存放与读取

冗余存储 HDFS 将文件切分为块, 每个块保存多个副本。副本通常分布在不同 DataNode, 甚至不同机架上, 以提高容错性和可用性。

写入过程 客户端向 NameNode 请求创建文件，NameNode 返回可写入的 DataNode 列表。客户端将数据按块写入第一个 DataNode，并通过管道复制到其他副本节点。写入成功后，DataNode 向客户端和 NameNode 汇报状态。

读取过程 客户端向 NameNode 查询目标文件的块位置，NameNode 返回各块所在 DataNode。客户端优先选择距离近或负载合适的 DataNode 读取数据，并按块顺序组合成完整文件。

5.12 HDFS 如何实现错误恢复

HDFS 把硬件故障视为常态，通过以下机制实现错误检测和恢复：

- **心跳机制**：DataNode 定期向 NameNode 汇报状态，若长时间未汇报则被视为故障。
- **块报告**：DataNode 定期报告自身保存的数据块列表。
- **副本补齐**：当某些块副本数低于设定值时，NameNode 调度其他 DataNode 复制新的副本。
- **元数据备份**：FsImage 和 EditLog 可通过备份或检查点机制降低 NameNode 元数据损坏风险。
- **数据校验**：通过校验和发现数据损坏，并从其他副本恢复。

5.13 并行计算、集群计算、分布式计算的区别

- **并行计算**：强调多个处理单元同时执行任务，目标是提高计算速度。
- **集群计算**：强调由多台相对独立的计算机组成集群，对外提供统一计算能力。
- **分布式计算**：强调任务和数据分布在不同节点上，通过网络协同完成计算。

三者有交叉关系：分布式系统常运行在集群上，并利用并行计算思想提高处理效率。

5.14 MapReduce 基本思想及运作过程

MapReduce 基于“分而治之”：

1. 输入数据存放在 HDFS 中，并被划分为多个逻辑分片 Split。
2. 多个 Mapper 并行读取分片，生成中间键值对。
3. 框架执行 Shuffle，把相同键的数据分组并传递给 Reducer。
4. Reducer 对每组数据进行汇总、统计、排序或合并。
5. 最终结果写回 HDFS 或其他存储系统。

以 WordCount 为例, Map 阶段把每个单词映射为 $(word, 1)$, Shuffle 阶段按单词分组, Reduce 阶段对每个单词的 1 求和, 得到词频。

5.15 MapReduce 架构、组件及作用

MapReduce 采用 Master/Slave 架构, 主要组件包括:

- **Client:** 提交 MapReduce 程序, 并查询作业运行状态。
- **JobTracker:** 负责资源监控、作业调度、任务分配和失败转移。
- **TaskTracker:** 周期性向 JobTracker 发送心跳, 执行分配到本节点的任务。
- **Mapper:** 处理输入分片, 生成中间键值对。
- **Reducer:** 处理 Shuffle 后的中间结果, 生成最终输出。

JobTracker 与 TaskTracker 的区别在于: 前者负责全局调度和监控, 后者负责本节点任务执行。Mapper 与 Reducer 的区别在于: 前者负责局部映射和中间结果生成, 后者负责按键聚合和最终结果生成。

5.16 MapReduce 与 HDFS 的基本数据单元及区别

HDFS 的基本存储单位是 **Block**, 它是物理存储层面的数据块, 默认大小在早期 Hadoop 中常为 64MB, 后续版本常为 128MB。

MapReduce 的基本处理单位是 **Split**, 它是逻辑输入分片, 描述任务处理的数据范围和位置信息。

二者区别是: Block 面向存储, Split 面向计算。Split 通常与 Block 大小接近, 但不是完全等同的物理概念。

5.17 Spark 基本思想及主要特点

Spark 是基于内存计算的大数据并行计算框架, 适合构建大规模、低延迟的数据分析应用。它的基本思想是把中间结果尽量保存在内存中, 减少反复读写磁盘的开销, 并通过 DAG 调度优化任务执行。

Spark 的主要特点包括:

- **速度快:** 内存计算显著提高迭代计算和交互式分析效率。
- **表达能力强:** 提供比 MapReduce 更丰富的数据操作。
- **通用性强:** 支持批处理、流处理、机器学习、图计算和 SQL。
- **调度优化:** 基于 DAG 的任务调度机制减少不必要的中间落盘。

5.18 Spark 与 Hadoop 的关系

Spark 不是 Hadoop 的简单修改版，也不完全依赖 Hadoop。Spark 可以独立部署，也可以运行在 Hadoop 生态中。当 Spark 与 Hadoop 结合时，常使用 HDFS 作为存储，并可通过 YARN 进行资源管理。

相对 Hadoop MapReduce, Spark 的优势在于更灵活、更高效、更适合迭代计算。Hadoop 的优势在于生态成熟、HDFS 稳定、批处理能力可靠。

5.19 Spark 架构、基本组件及其关系

Spark 应用的基本组件包括：

- **Driver**: 应用程序的主控进程，负责创建 SparkContext、生成任务、调度执行并收集结果。
- **Cluster Manager**: 集群资源管理器，如 Standalone、YARN 等。
- **Executor**: 运行在工作节点上的进程，负责执行 Task 并缓存数据。
- **Task**: 具体执行的数据处理任务。
- **Application**: 用户提交的一次 Spark 应用。

运行时，Driver 向集群管理器申请资源，启动 Executor，然后把任务发送到 Executor 执行。执行结果可返回 Driver，也可写入 HDFS 或数据库。

6 第六章：数据可视化

6.1 DIKW 体系

DIKW 体系描述数据、信息、知识和智慧之间的递进关系：

- **Data, 数据**: 原始事实和记录，如车速、流量、位置点。
- **Information, 信息**: 经过整理后具有意义的信息，如某路段平均速度下降。
- **Knowledge, 知识**: 从信息中总结出的规律，如早高峰拥堵与通勤流量集中有关。
- **Wisdom, 智慧**: 基于知识进行判断和决策，如调整信号配时或发布绕行诱导。

数据可视化的重要作用就是帮助人从数据和信息中更快识别模式、形成知识并支持决策。

6.2 可视化定义

数据可视化是利用图形、图像、颜色、空间位置、动画和交互等视觉表达方式，将数据及其关系转化为人容易理解的形式。它不仅是“画图”，更是通过视觉编码帮助用户发现规律、比较差异、理解结构和支持决策。

6.3 数据可视化作用

数据可视化的主要作用包括：

- **分析数据和展示特征：**帮助识别趋势、分布、相关性和异常值。
- **追踪数据动态：**展示实时路况、实时用户活动和交通状态变化。
- **辅助理解数据：**降低复杂数据的认知难度。
- **增强表达效果：**提高数据报告、管理平台和决策系统的吸引力与可读性。

6.4 数据可视化基本原则

好的可视化应满足“信、达、雅”：

- **信, Faithfulness:** 准确表达数据，不产生误导、偏差和歧义。要控制谎言因子，防止图形变化夸大或缩小数据变化。
- **达, Expressiveness:** 高效、清晰地传达信息。要提高数据墨水占比，减少无效装饰，并利用合适的符号和视觉通道。
- **雅, Elegance:** 美观、协调、有设计感，使受众愿意阅读并能保持良好体验。

高效表达还应考虑格式塔原则，如邻近、相似、连续、对称、连通等，帮助用户快速识别群组关系。

6.5 几种可视化技术

- **基于几何的可视化：**使用折线图、柱状图、直方图、饼图、散点图、箱线图、平行坐标、散点图矩阵等表达数据。
- **基于像素的可视化：**把每个数据项映射为带颜色的像素，适合展示大规模高维数据。
- **基于图标的可视化：**用图标的大小、长度、形状、颜色等属性表示多维数据，如星形图、Chernoff 面法。
- **基于层次的可视化：**展示层级结构和网络关系，如节点链接图、树图、邻接矩阵和混合可视化。
- **地图可视化：**展示数据在地理空间上的分布特征与规律，如交通流量热力图、拥堵地图、轨迹地图。
- **动态与交互式可视化：**通过动画展示数据变化，通过选择、缩放、定位、识别、路径绘制等交互操作增强探索能力。

7 综合简答题答题提示

闭卷简答题通常不要求把所有细节写满，而要抓住“定义、组成、流程、区别、作用、例子”。答题时可按如下模板组织：

1. 如果问“是什么”，先给定义，再补充核心特征和交通例子。
2. 如果问“区别”，分别说明对象 A 和对象 B 的职责、层次或数据单元，再用一句话总结。
3. 如果问“流程”，按步骤写清输入、处理、输出和反馈。
4. 如果问“作用”，从技术作用和交通应用价值两方面回答。
5. 如果问“关系”，说明各技术在采集、传输、存储、计算、分析、可信共享和决策中的分工。